

Managing Storage

Kubernetes Production · Lesson 9

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Distinguish **ephemeral** from **persistent** volumes and mount them into Pods
- Use **emptyDir**, **projected** sources, and understand **hostPath** caveats
- Explain the **PV / PVC** abstraction and its full **lifecycle**
- Choose the right **access modes** and **reclaim policies**
- Use **StorageClasses** for **dynamic provisioning** and tune `volumeBindingMode`
- Connect storage to **StatefulSet** `volumeClaimTemplates`

References: *Volumes*, *Persistent Volumes*, *Storage Classes* — kubernetes.io/docs/concepts/storage

9.1 Volumes

What Is a Volume?

A **volume** is a directory, possibly containing data, **accessible to the containers in a Pod**. Declared at the **Pod level** and mounted into containers.

- **Ephemeral** volumes have the **lifetime of the Pod** — destroyed when the Pod ceases to exist
- **Persistent** volumes exist **beyond** any individual Pod
- For *any* type, data is **preserved across container restarts** within the Pod

Two fields wire it together:

- `spec.volumes[]` — declares the volume on the Pod
- `containers[].volumeMounts[]` — mounts it at a `mountPath`

A volume **cannot be mounted inside another volume** — use `subPath` instead.

Mounting a Volume Into a Container

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: cache
          mountPath: /tmp/cache      # where the container sees it
        - name: cache
          mountPath: /var/log
          readOnly: true
  volumes:
    - name: cache                  # the volume definition
      emptyDir: {}
```

- The **same** volume can mount at **different paths**, even read-only
- Each container independently declares **where** to mount each volume

emptyDir — Scratch Space

An `emptyDir` is created **empty** when the Pod is **assigned to a node** and lives as long as that Pod runs on the node.

- **Survives container crashes**; deleted **permanently** when the Pod is removed
- Great for scratch space, checkpointing, sharing files between containers
- `sizeLimit` caps capacity; `medium: Memory` makes it a **RAM-backed tmpfs**

```
volumes:  
  - name: cache-volume  
    emptyDir:  
      sizeLimit: 500Mi  
      medium: Memory      # tmpfs — counts against the container's memory
```

CKA tip: `medium: Memory` files are RAM-backed and **charged against the Pod's memory limit** — not durable storage.

Projected Volumes

A **projected** volume maps **several sources into one directory**. Supported sources:

- **configMap** — non-sensitive configuration data
- **secret** — sensitive data (keys, tokens, passwords)
- **downwardAPI** — Pod / container metadata as files
- **serviceAccountToken** — short-lived, audience-scoped SA token

```
volumes:  
  - name: all-in-one  
    projected:  
      sources:  
        - configMap: { name: my-config }  
        - secret: { name: my-secret }  
        - downwardAPI:  
            items:  
              - path: "labels"  
                fieldRef: { fieldPath: metadata.labels }  
        - serviceAccountToken:  
            audience: vault  
            expirationSeconds: 3600  
            path: token
```

configMap, secret & downwardAPI

- **configMap** and **secret** volumes are mounted **read-only**; the ConfigMap / Secret must **exist before** the Pod uses it
- **downwardAPI** exposes Pod metadata (labels, annotations, resource limits) as files
- Updates to the source **propagate** to the mounted files automatically...
- ...**except** when mounted via **subPath** — those do **not** receive updates

```
volumes:  
  - name: config-vol  
    configMap:  
      name: log-config  
      items:  
        - key: log_level  
          path: log_level.conf      # appears at mountPath/log_level.conf
```

CKA tip: prefer projected/Secret volumes over env vars for credentials — they can be **rotated** without restarting the Pod.

hostPath — Handle With Care

`hostPath` mounts a file or directory **from the host node** into the Pod.

- `type` validates the target: `Directory`, `DirectoryOrCreate`, `File`, `FileOrCreate`, `Socket`, `CharDevice`, `BlockDevice`, or `""` (no checks)

```
volumes:  
  - name: host-volume  
    hostPath:  
      path: /data  
      type: Directory
```

Security warning: `hostPath` grants direct node filesystem access and enables privilege escalation / container escape. Avoid it; use a **PersistentVolume** instead.

9.2 PersistentVolumes & Claims

The PV / PVC Abstraction

Two API objects decouple **how** storage is provided from **how** it is consumed:

Object	Scope	Who owns it	Purpose
PersistentVolume (PV)	Cluster	Admin / dynamic provisioner	Storage with a lifecycle independent of any Pod
PersistentVolumeClaim (PVC)	Namespace	User / developer	A request for storage (size + access mode)

- A PVC binds to **one** PV in an exclusive **one-to-one** relationship (`claimRef`)
- Pods reference the **PVC**, never the PV directly

Think of it like Pods consuming **node** resources — PVCs consume **PV** resources.

Lifecycle: Provisioning

PVs come into existence in one of two ways:

- **Static** — an administrator pre-creates PVs with real storage details; they sit **Available** until a matching claim arrives
- **Dynamic** — when no static PV matches, the cluster provisions one **on demand** from the PVC's **StorageClass** (needs the `DefaultStorageClass` admission controller for default-class behaviour)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data
```

Lifecycle: Binding & Using

Binding

- A control loop matches a PVC to a suitable PV (capacity $\geq$ request, compatible access mode and StorageClass); the user gets **at least** what was requested
- If no PV matches, the PVC stays **Pending** indefinitely

Using

- The Pod mounts the PVC; **Storage Object in Use Protection** finalizers (`pvc-protection` / `pv-protection`) block deletion while in use

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
  storageClassName: manual
```

A Pod Consuming a PVC

A Pod references a PVC by name in `spec.volumes[].persistentVolumeClaim.claimName`.

```
apiVersion: v1
kind: Pod
metadata:
  name: task-pod
spec:
  containers:
    - name: task-container
      image: nginx
      volumeMounts:
        - mountPath: /usr/share/nginx/html
          name: task-storage
  volumes:
    - name: task-storage
      persistentVolumeClaim:
        claimName: task-pvc      # the PVC, in the same namespace
```

- PVCs are **namespace-scoped** — the Pod and PVC must share a namespace
- The bound PV may be **larger** than requested; capacity is a minimum

Access Modes

A PV advertises modes; a PVC **requests** the modes it needs.

Mode	Short	Meaning
ReadWriteOnce	RWO	Mounted read-write by a single node (many Pods on it OK)
ReadOnlyMany	ROX	Mounted read-only by many nodes
ReadWriteMany	RWX	Mounted read-write by many nodes
ReadWriteOncePod	RWOP	Mounted read-write by a single Pod (stable v1.29)

- Modes are about **node**-level mounting, not Pod count (except RWOP)
- A volume can only be mounted using **one** access mode at a time

CKA tip: most block storage (cloud disks, RBD) is **RWO**; shared filesystems (NFS, CephFS) provide **RWX**.

Lifecycle: Reclaiming

When a PVC is deleted, the PV's **reclaim policy** decides its fate:

Policy	Behaviour
Retain	PV becomes Released ; data kept, admin must reclaim manually
Delete	PV and the backing external storage are deleted automatically
Recycle	Basic scrub (<code>rm -rf /thevolume/*</code>) — deprecated

PV phases you will observe:

- **Available** → unbound · **Bound** → claimed · **Released** → claim gone, not reclaimed · **Failed** → reclaim error

CKA tip: with **Retain**, a Released PV is **not** automatically reusable — you must delete and recreate it (or clear its `claimRef`).

9.3 StorageClasses & Dynamic Provisioning

What Is a StorageClass?

A **StorageClass** describes a *class* of storage the cluster offers — a "profile" for a QoS tier or admin policy — and enables **dynamic provisioning**.

Key fields:

- **provisioner** (*required*) — the volume plugin / CSI driver to use
- **parameters** — provider-specific options passed to the provisioner
- **reclaimPolicy** — **Delete** (default) or **Retain** for provisioned PVs
- **allowVolumeExpansion** — permit growing provisioned volumes (no shrink)
- **mountOptions** — mount flags applied to provisioned PVs
- **volumeBindingMode** — *when* binding / provisioning happens

```
kind: StorageClass lives under apiVersion: storage.k8s.io/v1 .
```

StorageClass Manifest

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Retain
allowVolumeExpansion: true
mountOptions:
  - discard
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: ssd
```

- A PVC selects a class via `spec.storageClassName: fast-ssd`
- Omit it and the **default** StorageClass (if any) is used

volumeBindingMode

Controls **when** the volume is bound and (for dynamic) provisioned:

Mode	When binding happens
Immediate <i>(default)</i>	As soon as the PVC is created
WaitForFirstConsumer	Delayed until a Pod using the PVC is scheduled

- **Immediate** can provision storage in a zone the Pod's node **cannot reach**
- **WaitForFirstConsumer** provisions respecting the Pod's **resource requests, node selectors, affinity, taints/tolerations** — the topology-aware choice

CKA tip: for zonal cloud disks, prefer **WaitForFirstConsumer** so the disk lands in the same zone as the Pod (don't use **nodeName** with it).

The Default StorageClass

- Mark exactly **one** class default with the annotation
`storageclass.kubernetes.io/is-default-class: "true"`
- A PVC that **omits** `storageClassName` then uses that default automatically
- Setting `storageClassName: ""` (empty string) **disables dynamic provisioning** for that PVC — it can bind only to a matching pre-created PV

```
kubectl get storageclass  
kubectl get sc -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-class=="true")].metadata.name}'
```

Keep only **one** default — if several exist, the **most recently created** wins.

Storage in StatefulSets

StatefulSets give each replica **stable, per-Pod storage** via `volumeClaimTemplates` — a PVC template, not a shared claim.

```
spec:
  serviceName: web
  replicas: 3
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ "ReadWriteOnce" ]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 1Gi
```

- One PVC is created **per Pod** (e.g. `data-web-0`, `data-web-1`, ...)
- Claims (and their PVs) **persist** across rescheduling and are **not** deleted on scale-down

CKA tip: deleting a StatefulSet leaves its PVCs behind — clean them up manually.

Key Takeaways

- Volumes attach at the **Pod** level (`spec.volumes`) and mount via `volumeMounts`
- **emptyDir** is ephemeral scratch; **projected** combines configMap/secret/downwardAPI/SA-token; **hostPath** is a security risk — avoid it
- **PV** (cluster) + **PVC** (namespace) decouple provisioning from consumption; lifecycle is **provision** → **bind** → **use** → **reclaim**
- Know the access modes: **RWO**, **ROX**, **RWX**, **RWOP**; reclaim policies **Retain** / **Delete**
- **StorageClass** drives **dynamic provisioning**; **WaitForFirstConsumer** is topology-aware; one **default** class per cluster
- **StatefulSets** use **volumeClaimTemplates** for stable per-Pod storage

End of Lesson 9

Next: Lesson 10 — Application Access

```
kubectl get pv · kubectl get pvc · kubectl get sc · kubectl describe pvc <name>
```